



University
of Exeter

COMM117

Large Language Models and Application

Greedy decoding (Deterministic Methods)

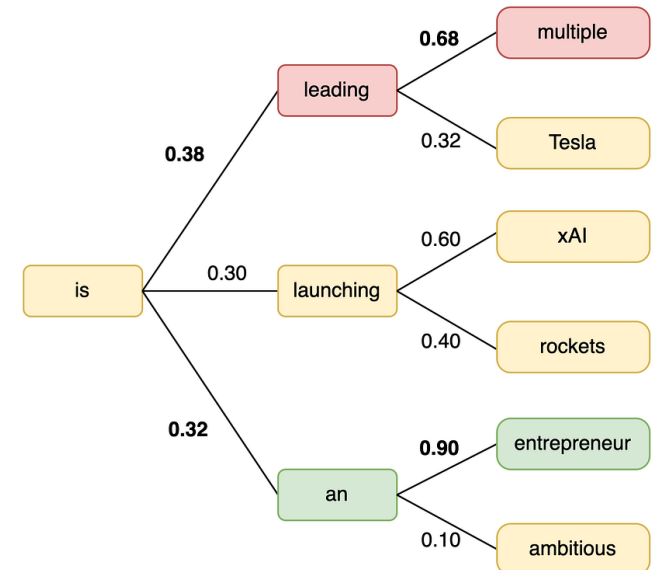
```
def autoregressive_decoding(x_seq, model, T):  
    n = len(x_seq[-1])  
    while n < T :  
        # get output probability distribution  
        q_next = get_probabilities(x_seq, model)  
        # sample next output token  
        x_next = sample(q_next)  
        # append next token to the input sequence  
        x_seq = torch.cat([x_seq, x_next], dim=-1)  
        n += 1  
  
    return x_seq
```

Pick the highest-prob token each step

- Fast, deterministic ✓
- Can be repetitive / get stuck in dull outputs ✗

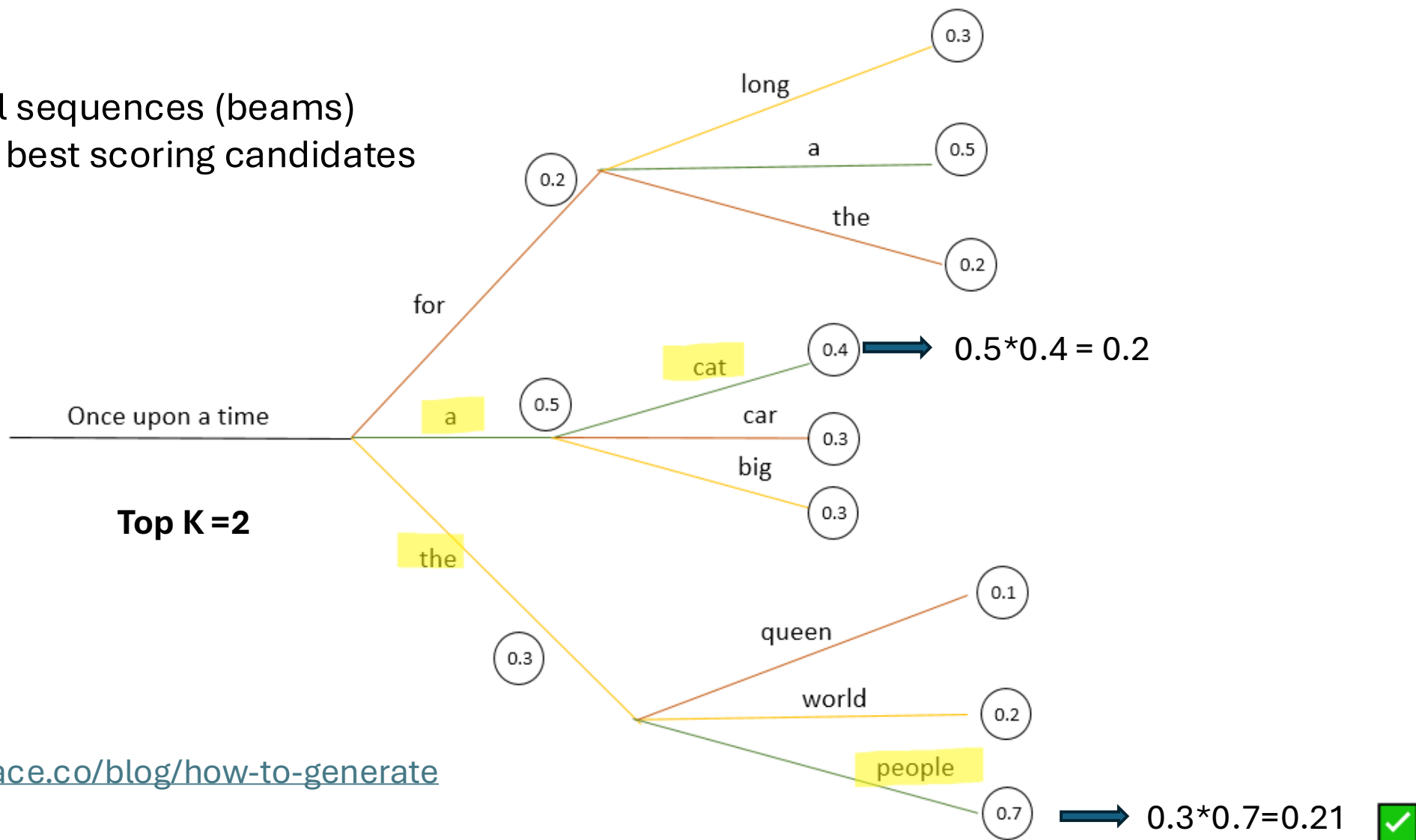
```
def sample(p):  
    return np.argmax(p)
```

Elon Musk



Beam Search

- Keep top **B** partial sequences (beams)
- Expand and keep best scoring candidates



Sampling

greedy search → terse, repetitive text

beam search → infeasible due to its high latency.

Goal: increase the creativity of our generations without sacrificing latency

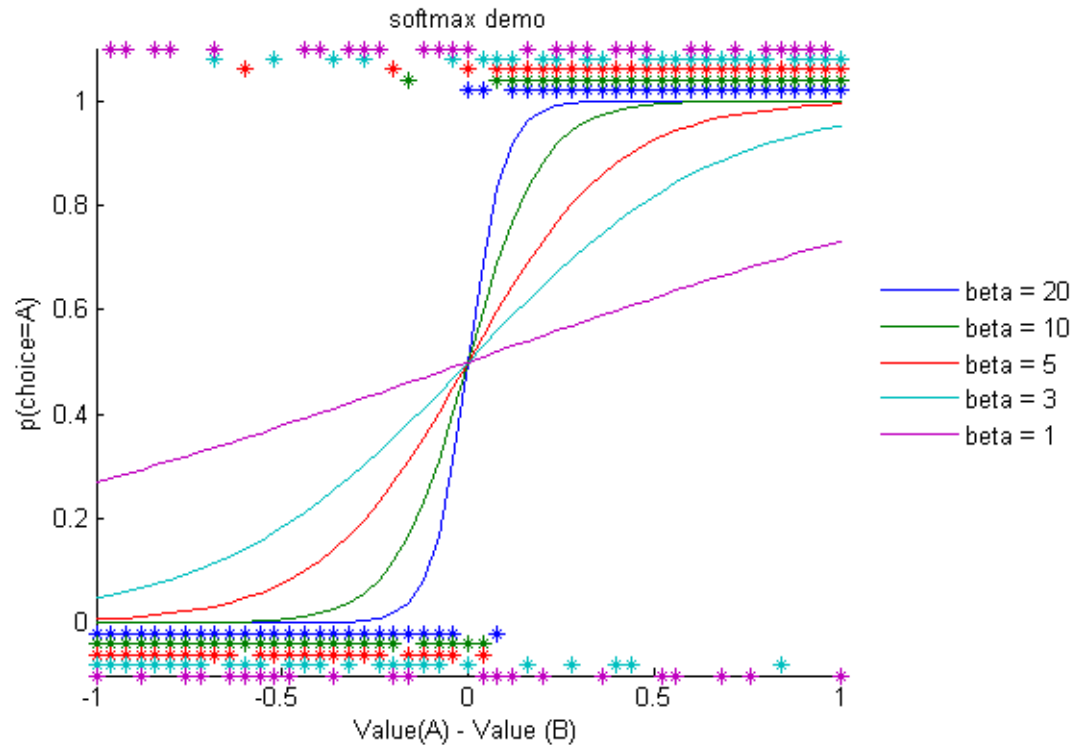
Non-deterministically sample the tokens to generate from the output distribution of the LLM.

Step 1:

Convert the LLM's output
Into probability distribution

```
probs = F.softmax(o) # Convert o, the logits, into probabilities  
token = torch.multinomial(probs, 1) # sample one token from  
the distribution
```

Sampling



`F.softmax(o)` # Convert o, the logits, into probabilities
`torch.multinomial(probs, 1)` # sample one token from
ribution

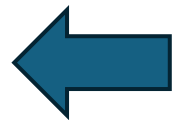
extremely **low probability** get generated!

$$p_i = \frac{\exp(l_i/T)}{\sum_j \exp(l_j/T)}$$

T close to zero

→ give more deterministic
outputs

Higher → More creativity



T=1 base softmax

Lower T Beta ↑ sharper probability distribution

Higher T Beta ↓ diffuse distribution

Sampling

But don't want the LLM to sample from the *entire* vocabulary

Motivation: Introduce some **randomness**, but don't really want to generate outlandish tokens.

Randomly sample the final token from **fixed size set**

→ **Top –K sampling**

However

- if the set size k was **too small**, the responses would be **repetitive and boring**,
- if **too large**, → lead to the **generation of outlandish tokens** from the ***tail of the probability distribution***.



Top – p sampling

Speculative decoding

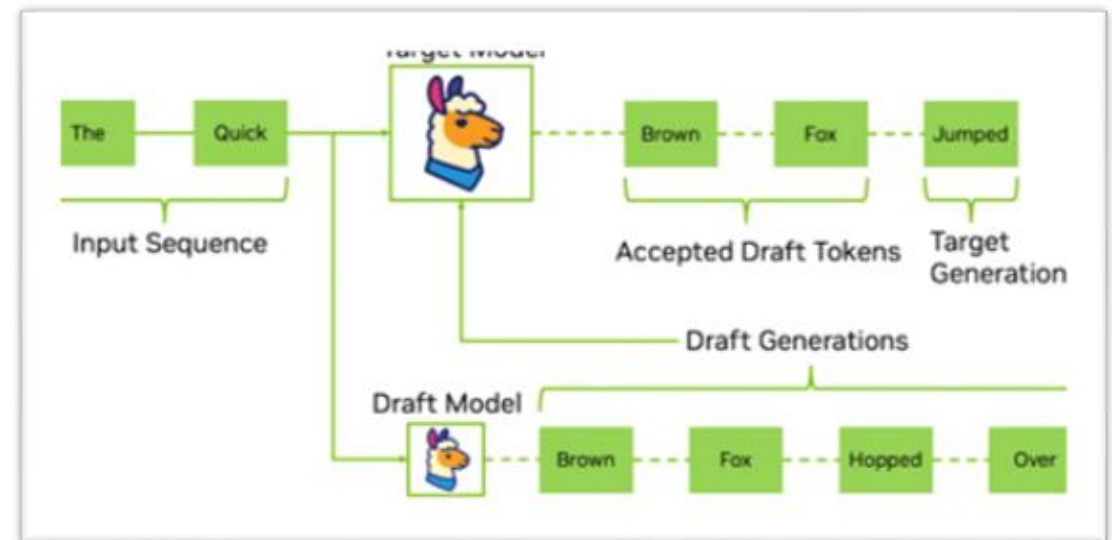
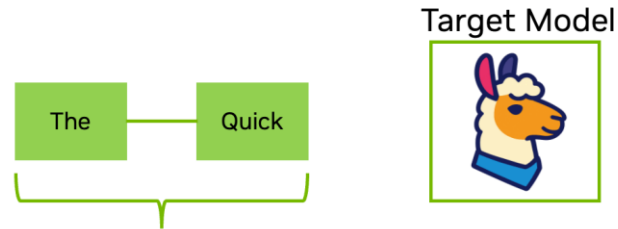
- Use a **small draft model** to propose multiple tokens (3-12 tokens)
→ small model trained on **same data distribution**
- Large model **verifies** instead of generating every token
- If many tokens are accepted → **fewer big-model forward passes**

[Accelerating Large Language Model Decoding with Speculative Sampling](#), Deep Mind

[Fast Inference from Transformers via Speculative Decoding](#)

Google research

<https://developer.nvidia.com/blog/an-introduction-to-speculative-decoding-for-reducing-latency-in-ai-inference/>

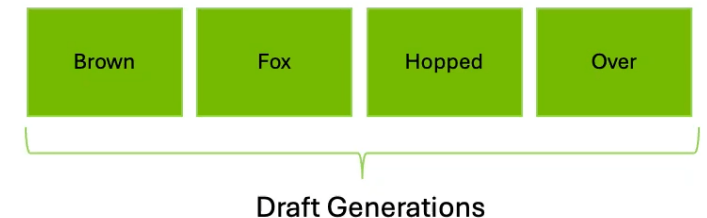


Speculative decoding- Reject sampling

Reject? Accept?

- If the **target model's predicted probability $q(x)$** is **higher** than the **draft model's $p(x)$** , **keep** the token.
- If not, then **with probability $1 - q(x)/p(x)$** , **replace the token** with a token sampled from $\max(0, q(x) - p(x))$.

- If token is **accepted**, it becomes part of the output.
- When you hit the **first rejected token**, you stop accepting further draft tokens.
- Then you generate the next token directly from the **target model** (and continue with a new draft block).



Speculative decoding

Helps most when:

- Draft model is much faster
- Large model verification is efficient
- Output is predictable enough that many tokens get accepted

Doesn't help much when:

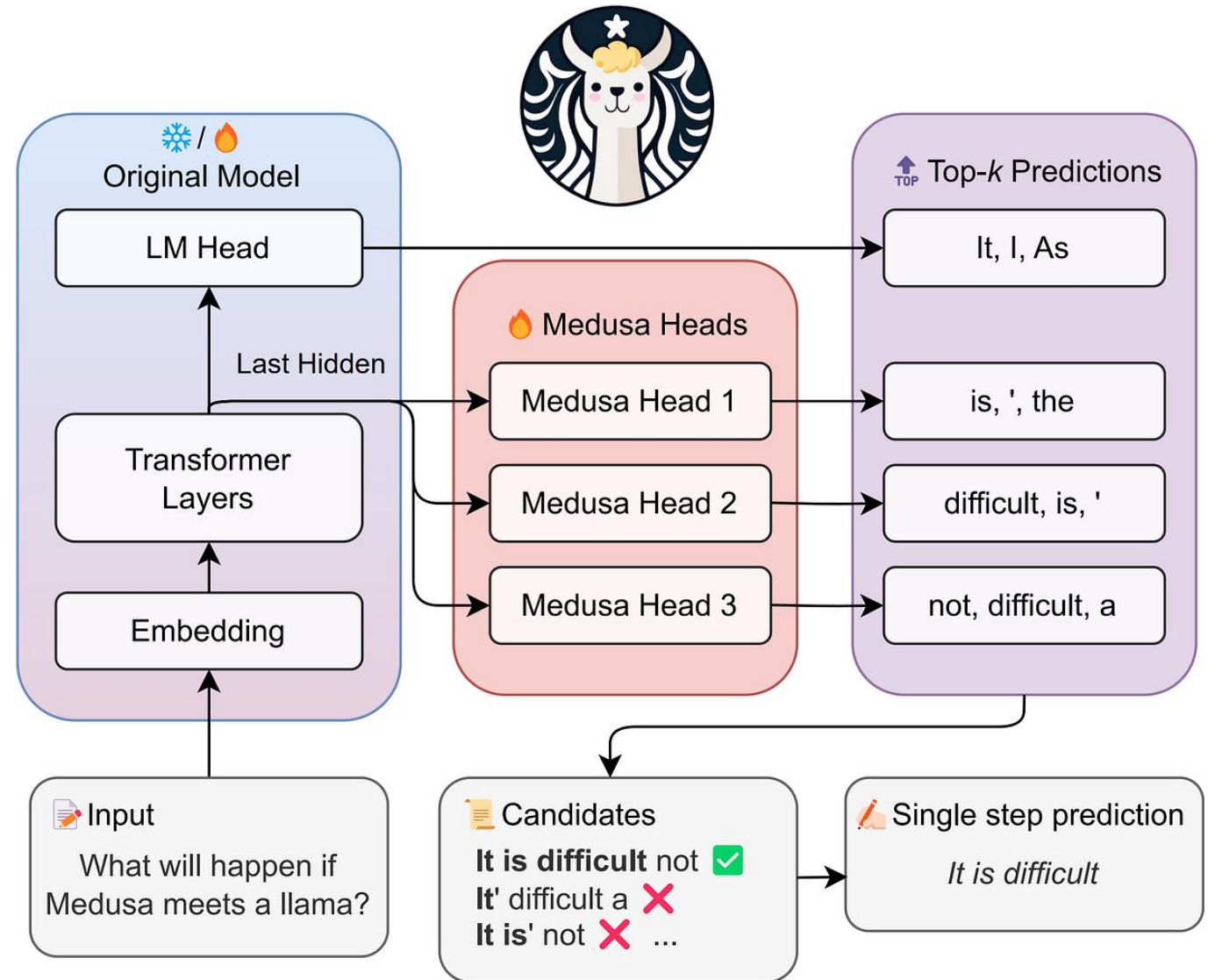
- Draft tokens are often rejected (very creative tasks, high temperature)
- Draft model is too weak / too slow

However...

The need for a separate draft model is too **cumbersome**. Can we do away with it?

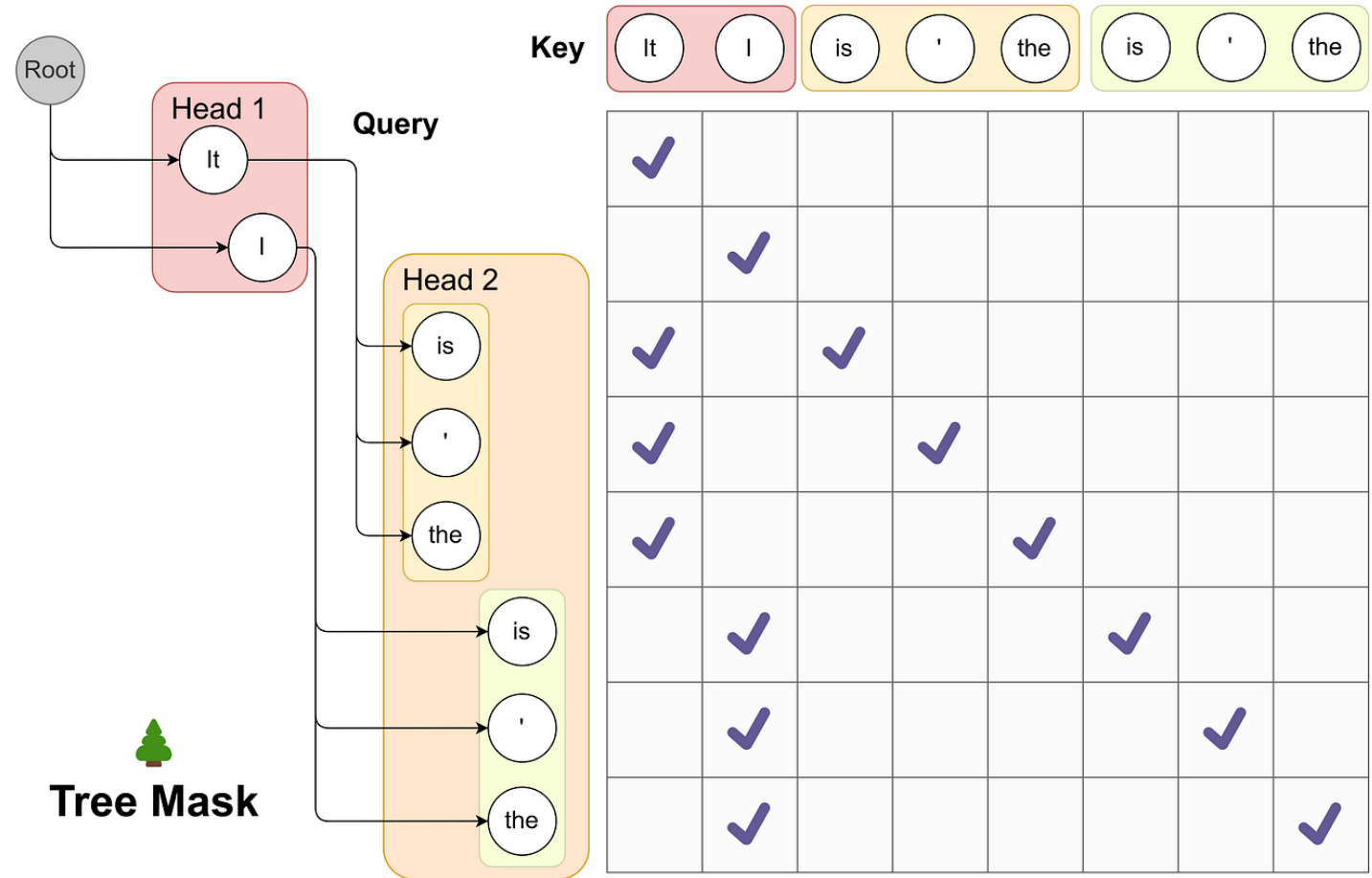
MEDUSA

- Instead of a separate draft model, you add **multiple “heads”** on the *same model* to propose **several future tokens** (a token “tree”) in one go.
- Then you **verify/accept** along a path.



MEDUSA

- **Add k additional LM heads to the transformer.** Specifically, we take the last **hidden state of the model**. Each LM head is a simple **2-layer FFN with a SiLU activation and residual connection** in the middle.
- **Each of the LM heads is finetuned.**
- **Keep track of the top s_k tokens generated by each head**
- **Tree attention**, to verify the entire tree of candidates at the same time.



MEDUSA- Tree Attention

Not just propose *one* next-token sequence. It proposes **multiple possible continuations** (branches) for a few steps ahead:

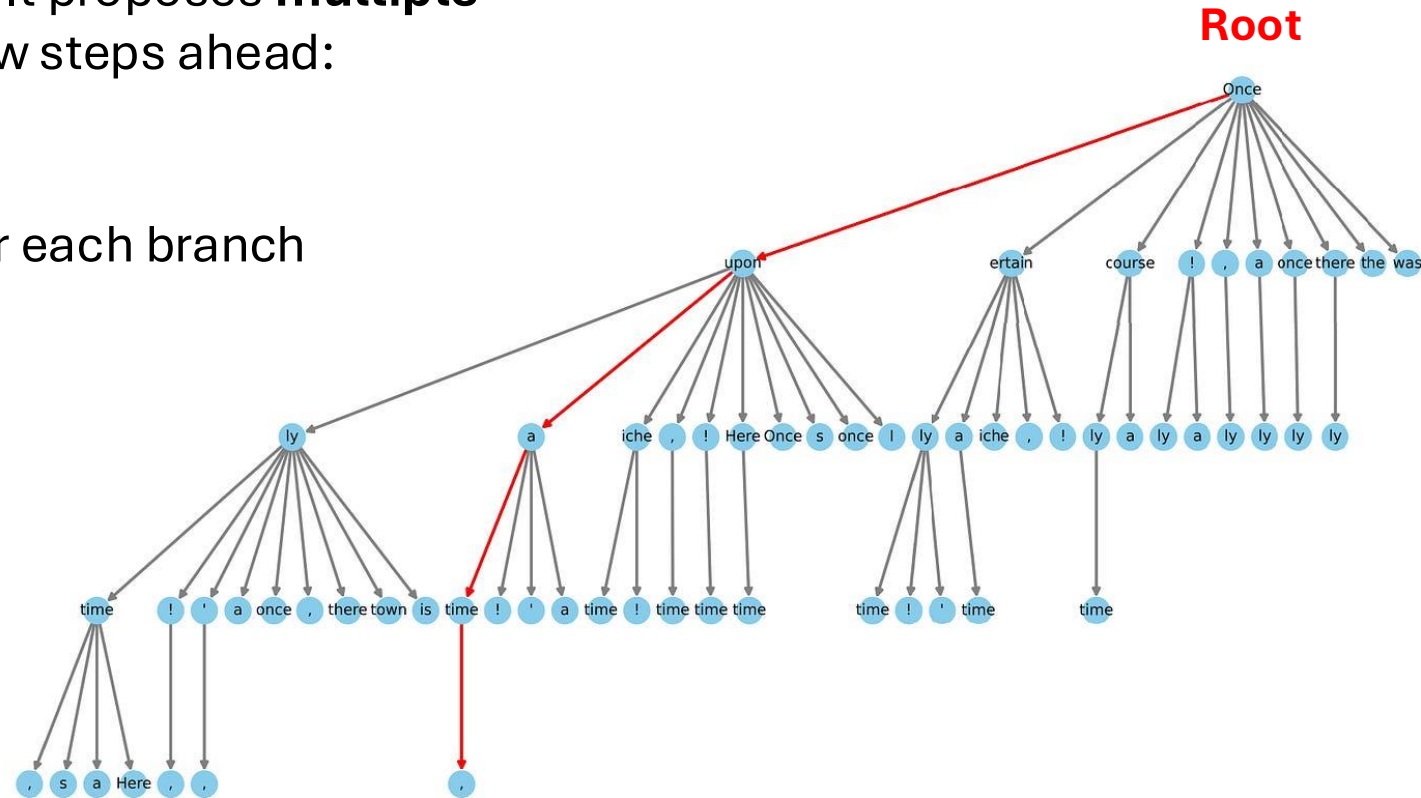
- root = your current prefix
- level 1 = several candidate next tokens
- level 2 = several candidate 2nd tokens for each branch
... forming a **token tree**

- **Shares computation for the common prefix**

- Stores **KV cache for the tree nodes**.

When evaluating a node, it attends to:

- the **original prefix tokens**, plus
- the **node's ancestors** in the tree (the path from root → that node)



MEDUSA



Introduction Today

1. Paper discussion: [EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty](#)
2. Evaluation Metrics for LLMs



EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty

Published as 2025.3.4

Motivation:

Autoaggressive decoding is time-consuming

Observation:

1. **Feature level** is more straightforward
2. **Inherent uncertainty** in feature level constrains the performance

of EAGLE, including all models from the Vicuna and LLaMA2-Chat series, the MoE model Mixtral 8x7B Instruct, and tasks in dialogue, code generation, mathematical reasoning, and instruction following. For LLaMA2-Chat 70B, EAGLE achieved a latency speedup ratio of **2.7x-3.5x**, doubled throughput, while maintaining the distribution of the generated text.

Abstract

Autoregressive decoding makes the inference of Large Language Models (LLMs) **time-consuming**. In this paper, we reconsider speculative sampling and derive two key observations. Firstly, autoregression at the feature (second-to-top-layer) level is more straightforward than at the token level. Secondly, the inherent uncertainty in feature (second-to-top-layer) level autoregression constrains its performance. Based on these insights, we introduce EAGLE (Extrapolation Algorithm for Greater Language-model Efficiency), **a simple yet highly efficient speculative sampling** framework. By incorporating a token sequence advanced by one time step, EAGLE effectively resolves the uncertainty, enabling precise second-to-top-layer feature prediction with minimal over-

We conducted comprehensive evaluations

Paper Discussion: EAGLE

- Unlike the draft-target approach, which relies on a separate draft model to propose tokens, EAGLE uses a **lightweight autoregressive prediction head collecting features from the target model's hidden states**.

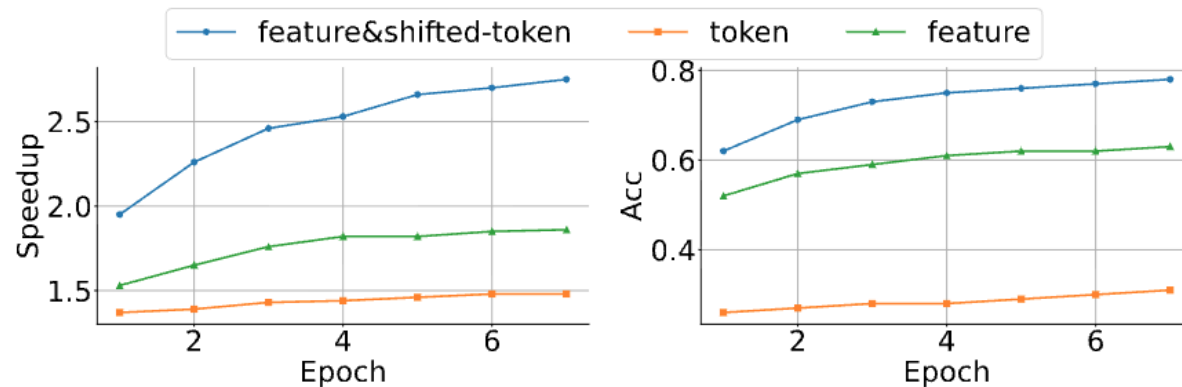
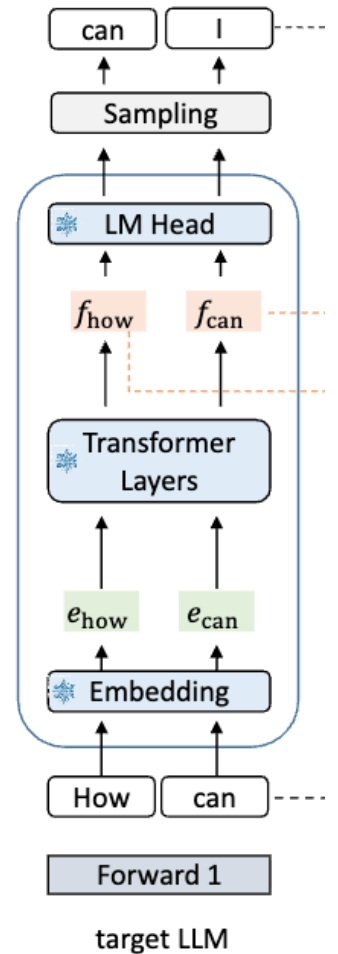


Figure 4: Accuracy and speedup ratio of draft models based on tokens, features and feature&shifted-token at tempera-

“**features**” refer to the **second-to-top-layer features of the original LLM**, located before the LM head.



Autoregressively processing at the feature level ; then **deriving tokens using the LM head of the original LLM** yields better results than directly autoregressively predicting tokens.

Paper Discussion: EAGLE

- The uncertainty inherent in the sampling process significantly constrains the performance of predicting the next feature.

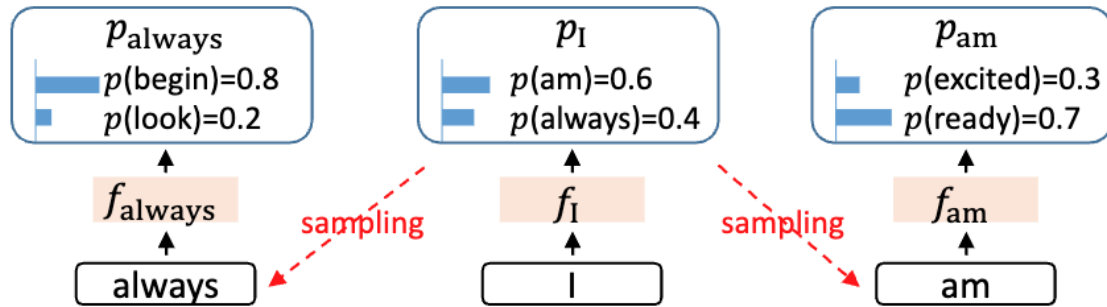
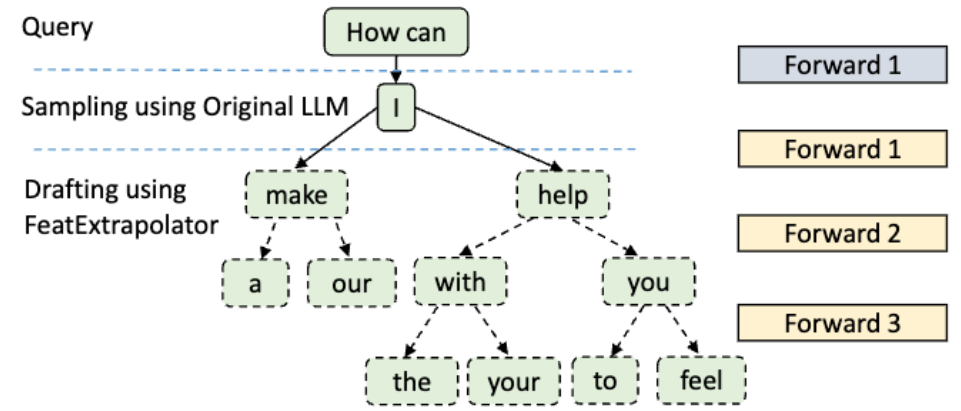
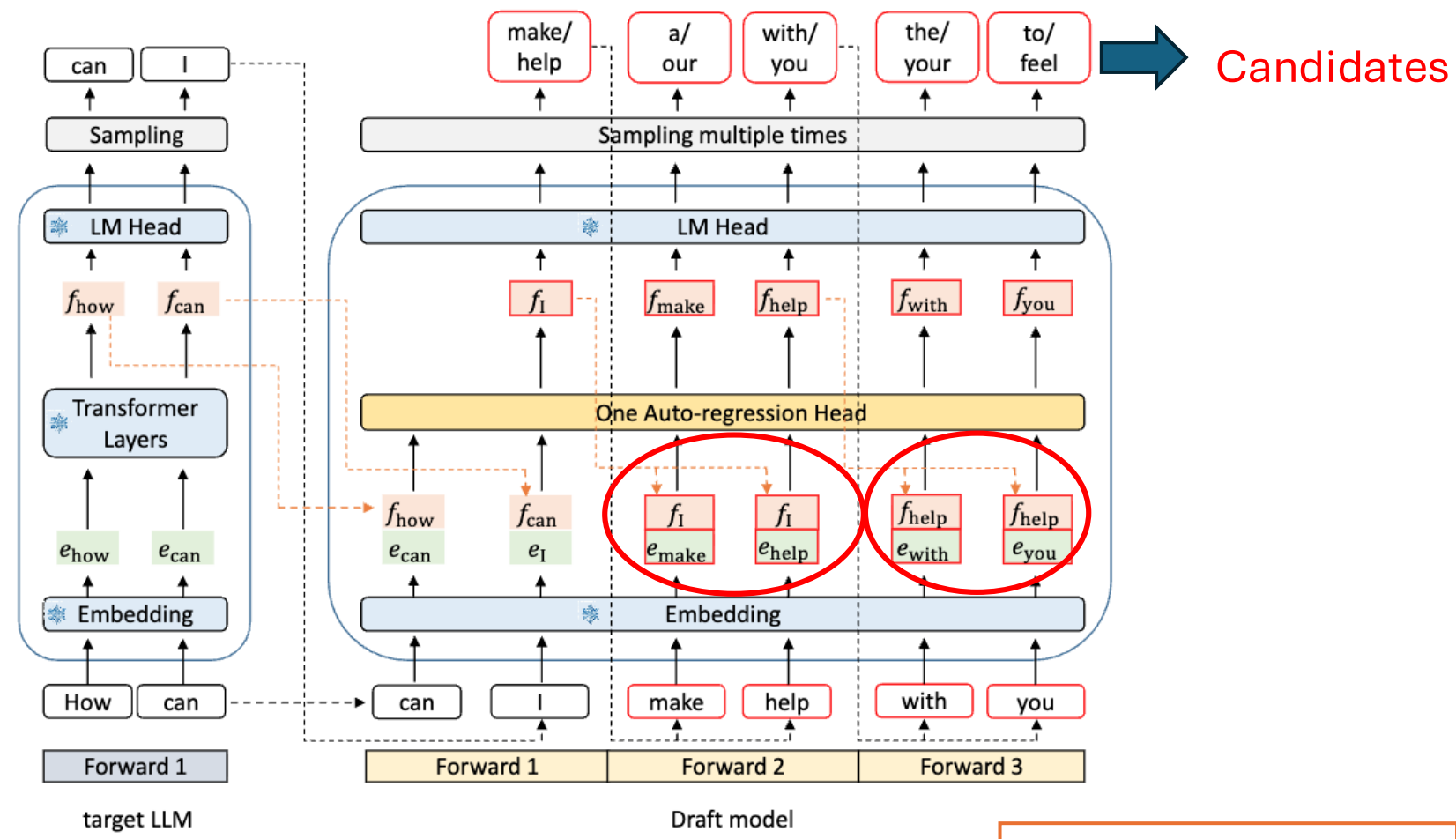
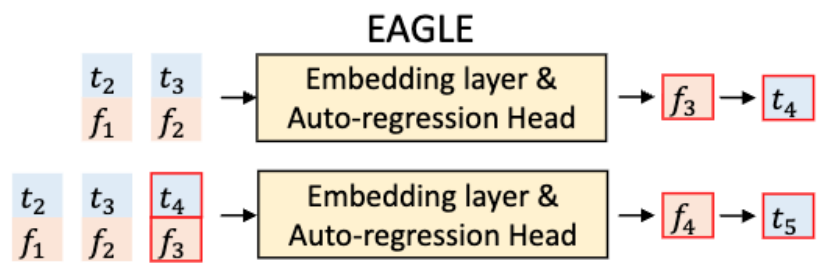
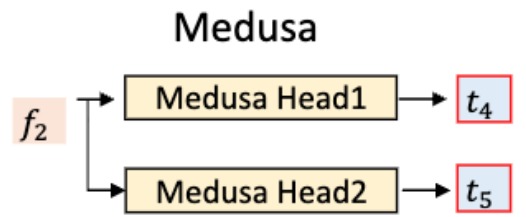
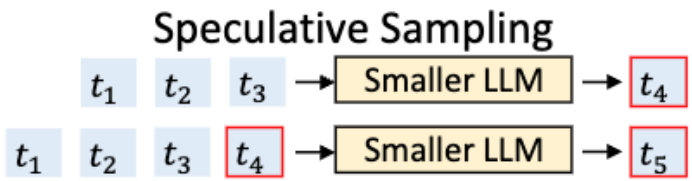


Figure 3: Uncertainty in feature sequences. The next feature following f_I is contingent on the sampling outcome and cannot be determined solely based on f_I , where both “always” and “am” are possible to follow the token “I” and lead to two branches.

- Predict next feature using only f_I , but the **next feature** could become f_{am} or f_{always} .
- EAGLE:** predict next feature using f_I and the **sampled next token** (one-step-ahead token)

Paper Discuss



Next: The **target LLM** **verifies** the drafted tokens **(accept/reject)** like standard speculative decoding.

Paper Discussion: Evaluation

Table 1: **Speedup ratio** and average acceptance length τ on HumanEval, GSM8K, and Alpaca. T denotes temperature, V represents Vicuna, and LC stands for LLaMA2-Chat.

		HumanEval		GSM8K		Alpaca	
	Model	Speedup	τ	Speedup	τ	Speedup	τ
T=0	V 7B	3.33x	4.29	3.01x	4.00	2.79x	3.86
	V13B	3.58x	4.39	3.08x	3.97	3.03x	3.95
	V 33B	3.67x	4.28	3.25x	3.94	2.97x	3.61
	LC 7B	3.17x	4.24	2.91x	3.82	2.78x	3.71
	LC 13B	3.76x	4.52	3.20x	4.03	3.01x	3.83
	LC 70B	3.52x	4.42	3.03x	3.93	2.97x	3.77

HumanEval (code),
GSM8K (math reasoning), and
Alpaca (instruction following).

Table 2: Average acceptance length τ and acceptance rate α on MT-bench. T denotes temperature.

	Model	τ	0- α	1- α	2- α	3- α	4- α
T=0	Vicuna 7B	3.94	0.79	0.74	0.72	0.73	0.67
	Vicuna 13B	3.98	0.79	0.74	0.72	0.74	0.70
	Vicuna 33B	3.68	0.74	0.69	0.67	0.67	0.66
	LLaMA2-Chat 7B	3.62	0.76	0.69	0.67	0.68	0.68
	LLaMA2-Chat 13B	3.90	0.77	0.69	0.69	0.70	0.71
	LLaMA2-Chat 70B	3.81	0.75	0.69	0.65	0.64	0.64

MT-Bench (chat)

Table 4: Generation speed of EAGLE combined with gpt-fast, evaluated on MT-bench with LLaMA2-Chat 7B at temperature=0.

Precision	FP16	int4
Vanilla (Huggingface)	24.5 tokens/s	N/A
gpt-fast	55.1 tokens/s	106.9 tokens/s
EAGLE + gpt-fast	100.2 tokens/s	160.4 tokens/s

Evaluation Matrix:

- **Speedup** vs vanilla decoding (how much faster)
- **Acceptance length / acceptance rate** (why speedup happens)
- **Throughput (tokens/sec)** and **latency** in a serving setting (case study)

It's trying to generate the **same distribution** as the target model **but faster**.

Evaluation

- “Better model” depends on **what you measure**
- Different metrics answer different questions
- Good eval = **quality + reliability + reproducibility**

How?

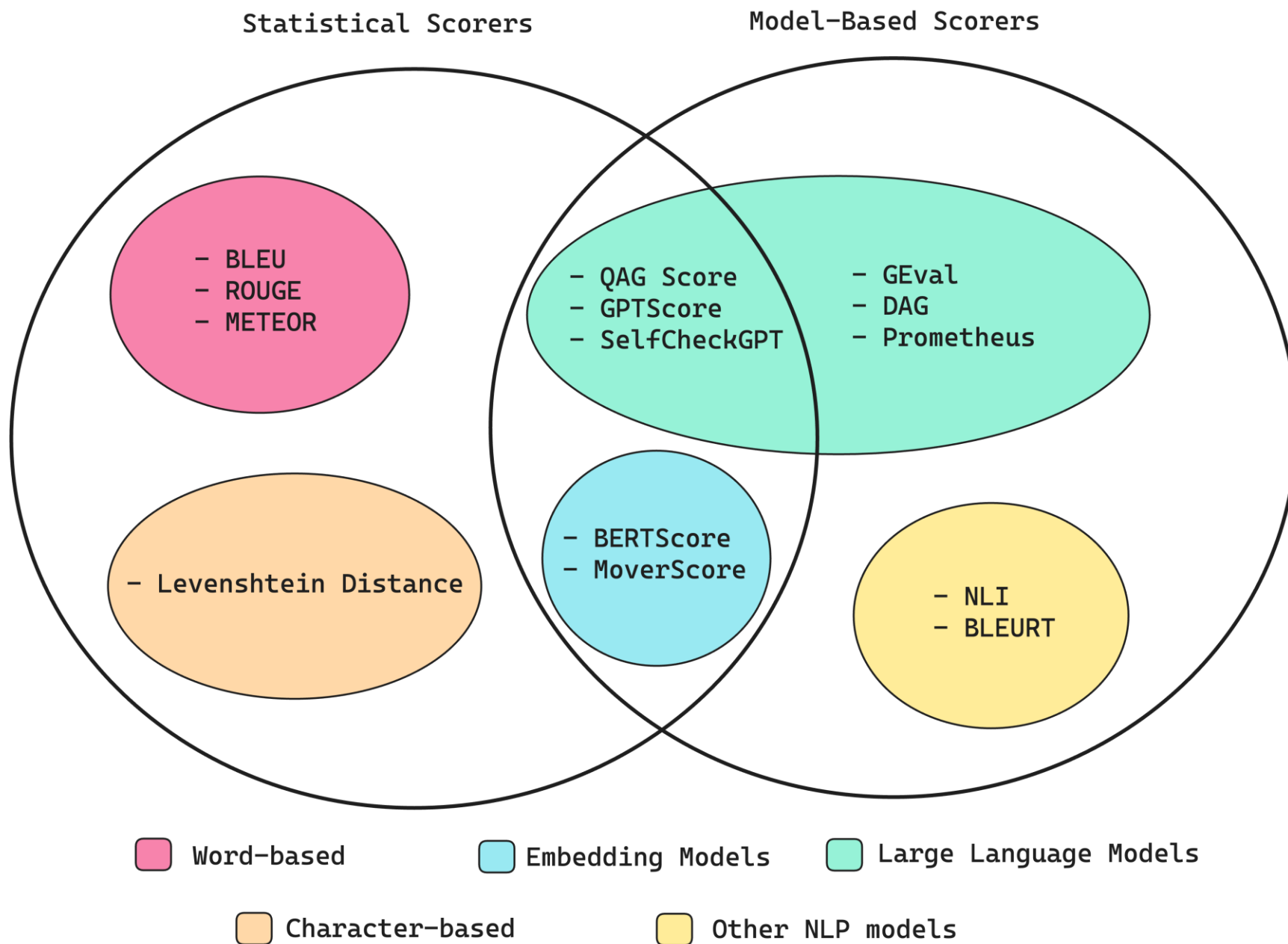
- **Benchmarks (automatic)** → meaning similarity / correctness
- **Human** as judge → preference, helpfulness, safety
- **System-level** → latency, throughput, cost, reliability

Eva

- “Bet
- Diffe
- Goc

Ho

- Ben
- Hun
- Syst



SS

Evaluation: Benchmarks (Automatic)

Automated benchmarking : you'd like to know **how well your model performs on something**.

- A well-defined task : *How well can my model classify spam from non spam emails?*
- More abstract and general capability: *How good is my model at math?*

- **Metric examples:** Accuracy/F1 (classification), EM/F1 (QA), pass@1 (code), ROUGE (summaries)
- **Dataset examples:** GSM8K (math), HumanEval (code), MMLU (general)

Benchmark scores $\neq$ real-world usefulness (prompt sensitivity, leakage, benchmark gaming)

Evaluation: Benchmarks (Automatic)

Common “Correctness” Metrics

- Classification: Accuracy, Precision/Recall, F1
- QA: Exact Match (EM), token F1
- Code: pass@k (unit tests)

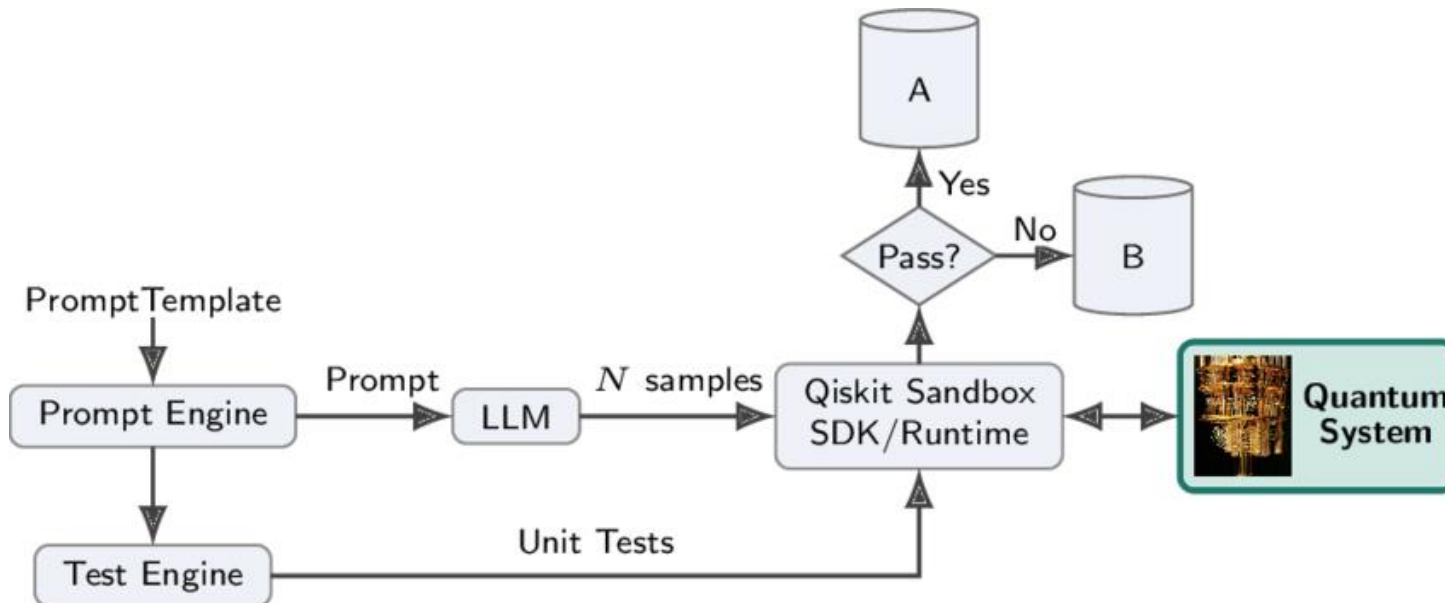
Exact Match (EM) Percentage of questions where the model’s final answer **matches the reference answer exactly** (after normalization like lowercasing/punctuation removal).

Token F1 Measures **overlap of tokens** between prediction and reference using
Precision = $\text{overlap} / \text{tokens in prediction}$
Recall = $\text{overlap} / \text{tokens in reference}$
F1 = harmonic mean of precision and recall

Evaluation: Benchmarks (Automatic)

Code: pass@k (unit tests)

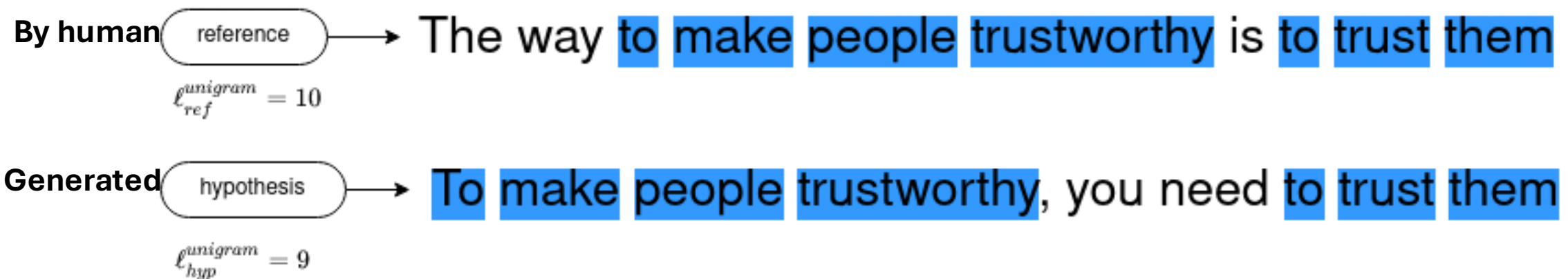
- **What it is:** For coding problems, you **sample k solutions** from the model and run unit tests.
pass@k = probability that **at least one** of the k samples passes all tests.
- **Why :** Code generation is **stochastic**; a model might produce a correct solution sometimes even if not always.



Evaluation: Benchmarks (Automatic)

Two of the most commonly used performance evaluation metrics

- **BLEU** (Bilingual Evaluation Understudy)
 - **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation) Scores.
- **BLEU Score:** measure the **precision** of n-grams in the machine-generated translation by comparing them to the reference **translations** : **how many words/phrases in the candidate also appear in the reference**



Evaluation: BLEU

By human

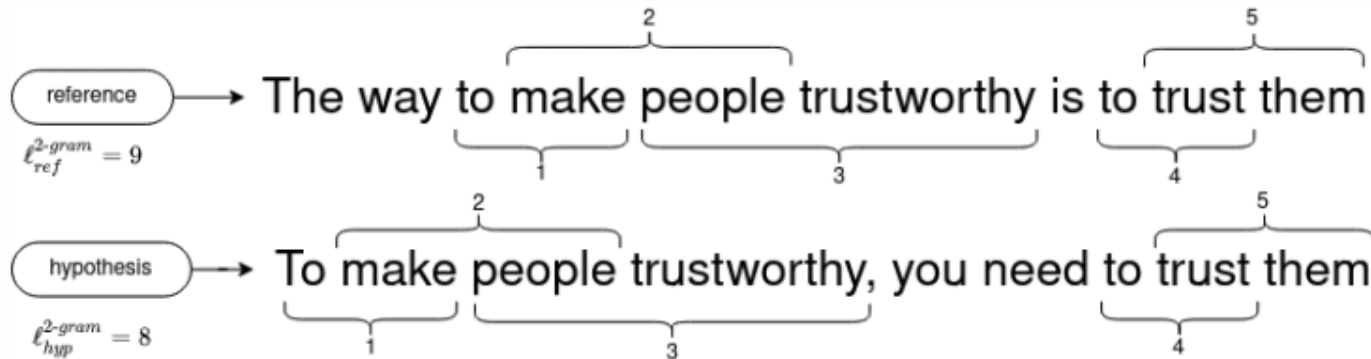
reference
 $\ell_{ref}^{unigram} = 10$

The way to make people trustworthy is to trust them

Generated

hypothesis
 $\ell_{hyp}^{unigram} = 9$

To make people trustworthy, you need to trust them



2-gram matches

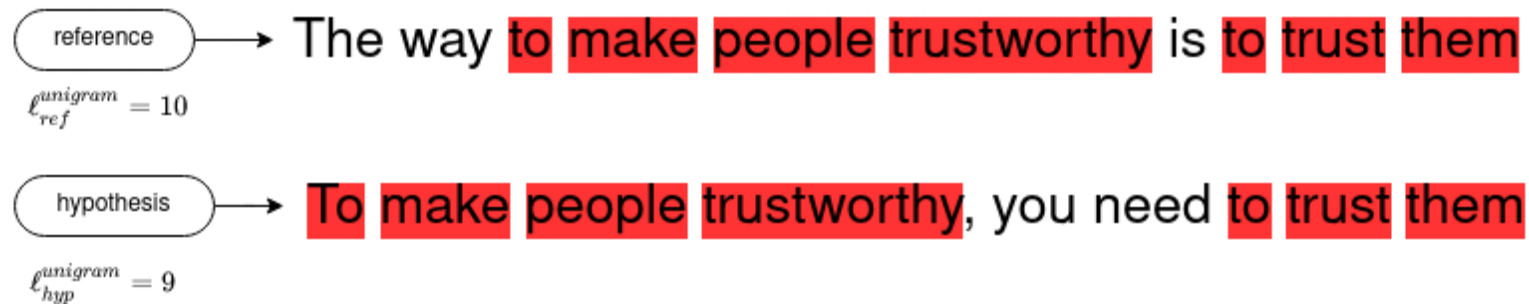
The following table details the precisions for 4 n-grams.

n-gram	1-gram	2-gram	3-gram	4-gram
p_n	$\frac{7}{9}$	$\frac{5}{8}$	$\frac{3}{7}$	$\frac{1}{6}$

Evaluation: Benchmarks (Automatic)

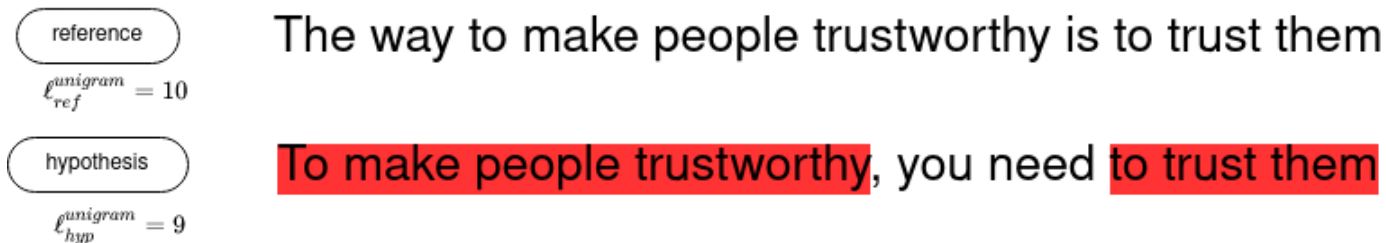
- **ROUGE Score:** It is specifically more focused on **recall**. It compares **overlapping units** like n-grams, words sequences, and word-pairs, in both generated text and the reference text. ROUGE scores commonly used for specific NLP tasks like **text summarization**.

ROUGE - 1



ROUGE-L

The length of the longest common subsequence (LCS)



$$\begin{cases} R_{LCS} &= \frac{7}{10} \\ P_{LCS} &= \frac{7}{9} \\ ROUGE_{LCS} &= \frac{(1+\beta^2)49}{70+\beta^2 63} \end{cases}$$

Combine Recall and precision score

Evaluation: Benchmarks (Automatic)

Dataset

GSM8K (Math)

- Grade-school math word problems
- Tests multi-step reasoning
- **Metric: accuracy** (exact final answer match; sometimes few-shot)

HumanEval (Code)

- Python programming tasks + unit tests
- Tests code generation correctness
- **Metric: pass@k** (probability at least 1 of k samples passes tests)

MMLU (General capability)

- **Multi-subject multiple-choice questions** (STEM, humanities, professional)
- Tests broad knowledge + reasoning
- **Metric: accuracy** (often 0-shot / few-shot)

Evaluation: Semantic evaluation

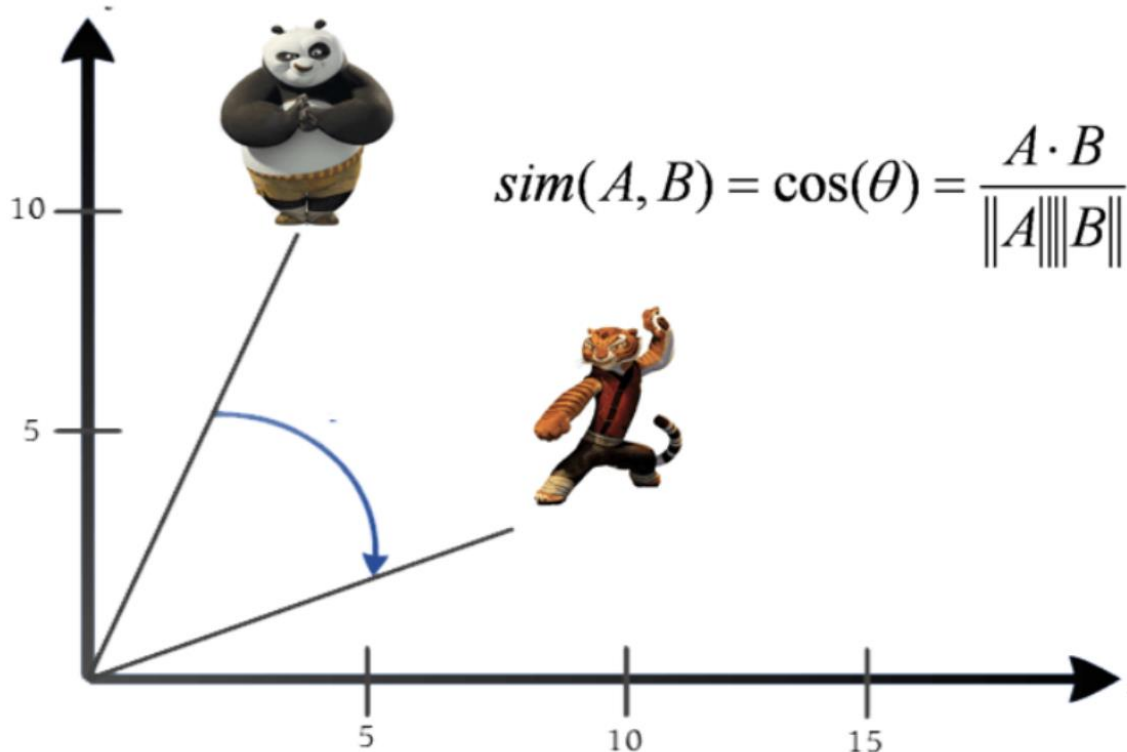
Semantic evaluation (automatic, meaning-aware)

- Embedding cosine similarity
- BERTScore
- QA-based checks (generate questions from reference, answer with candidate)

Evaluation: Semantic evaluation

- Embedding cosine similarity

Cosine Similarity



- Convert the reference and candidate texts into **vectors (embeddings)**.
- Compute **cosine similarity** between the vectors.
- High cosine  $\approx$ similar meaning (roughly).

Use : paraphrases / abstractive summaries where wording differs.

Limit : **can be fooled by fluent** but incorrect text if it “sounds similar.”

Evaluation: Semantic evaluation

QA-based checks

- Treat evaluation as: “Does the candidate preserve the key facts?”

Typical pipeline:

1. **Generate questions** from the reference (or gold answers).
2. **Answer** those questions using the **candidate summary**.
3. Score how often the **answers match the reference answers** (EM/F1).

Use: summarisation faithfulness/content coverage.

Limit: depends on the question generator/answering model; can introduce bias/errors.

Evaluation: Human evaluation

- **Pairwise preference**

A vs B, “which is better?”

Output: win-rate

- **Rubric scoring**

1–5 scale for: coverage, faithfulness, clarity, safety

Output: mean + variance

Evaluation: Human evaluation

Making human eval reliable

- Blind evaluation (hide model names)
- Multiple raters
- Measure agreement (κ)
- Balanced prompt set (not cherry-picked)
- Clear instructions + examples

Evaluation: LLM- as -judge

Use an LLM to **score or rank** model outputs using a rubric
Common forms:

Pairwise: A vs B $\rightarrow$ win-rate

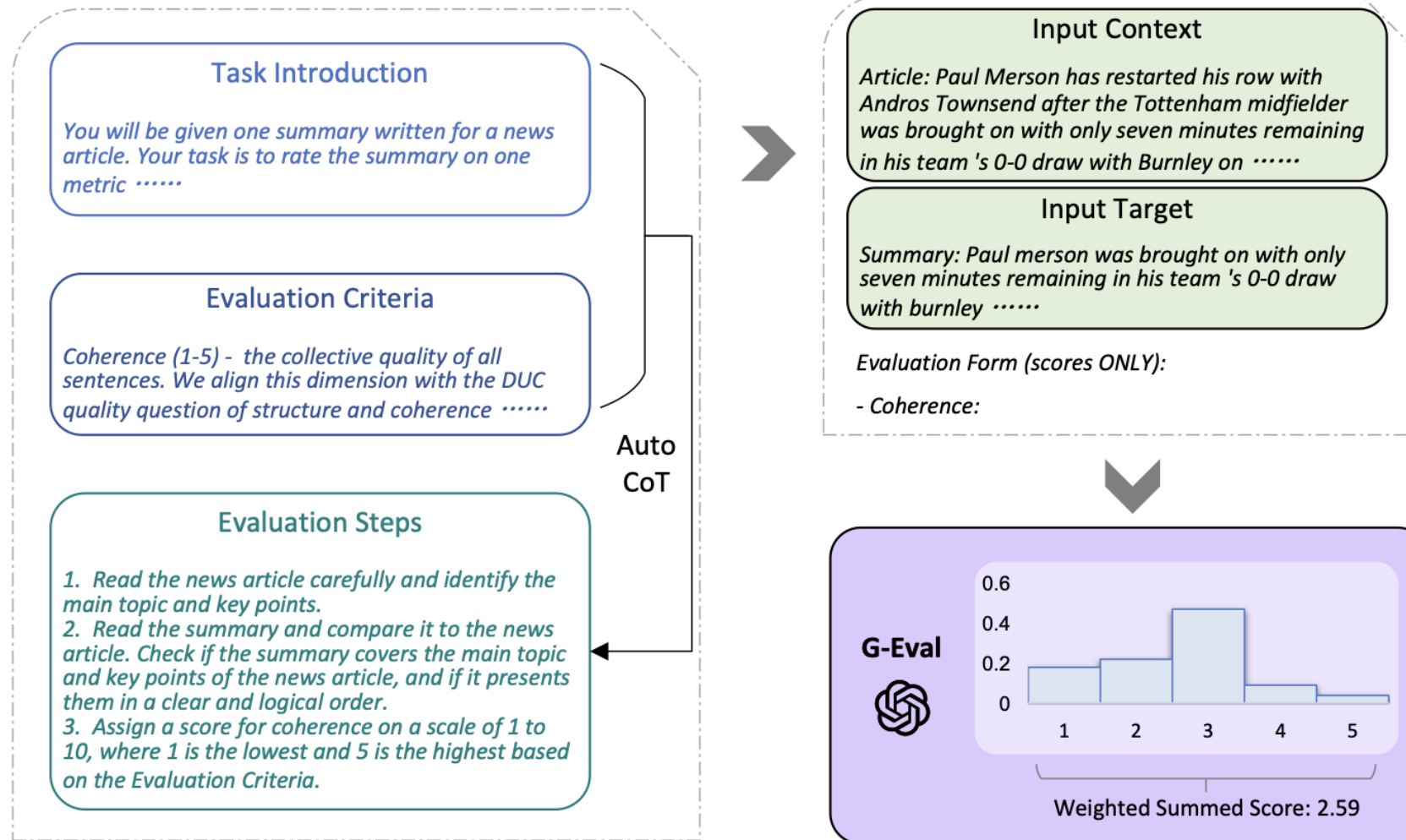
Rubric: helpfulness, faithfulness, safety (1–5)

Pros: cheap, fast, scalable

Risks: judge bias, prompt sensitivity, preference drift

Good practice: calibrate vs human labels + use multiple judges + report agreement

Evaluation: Model Based Score



G-EVAL: NLG Evaluation using GPT-4 with Better Human Alignment

Evaluation: QAG Score

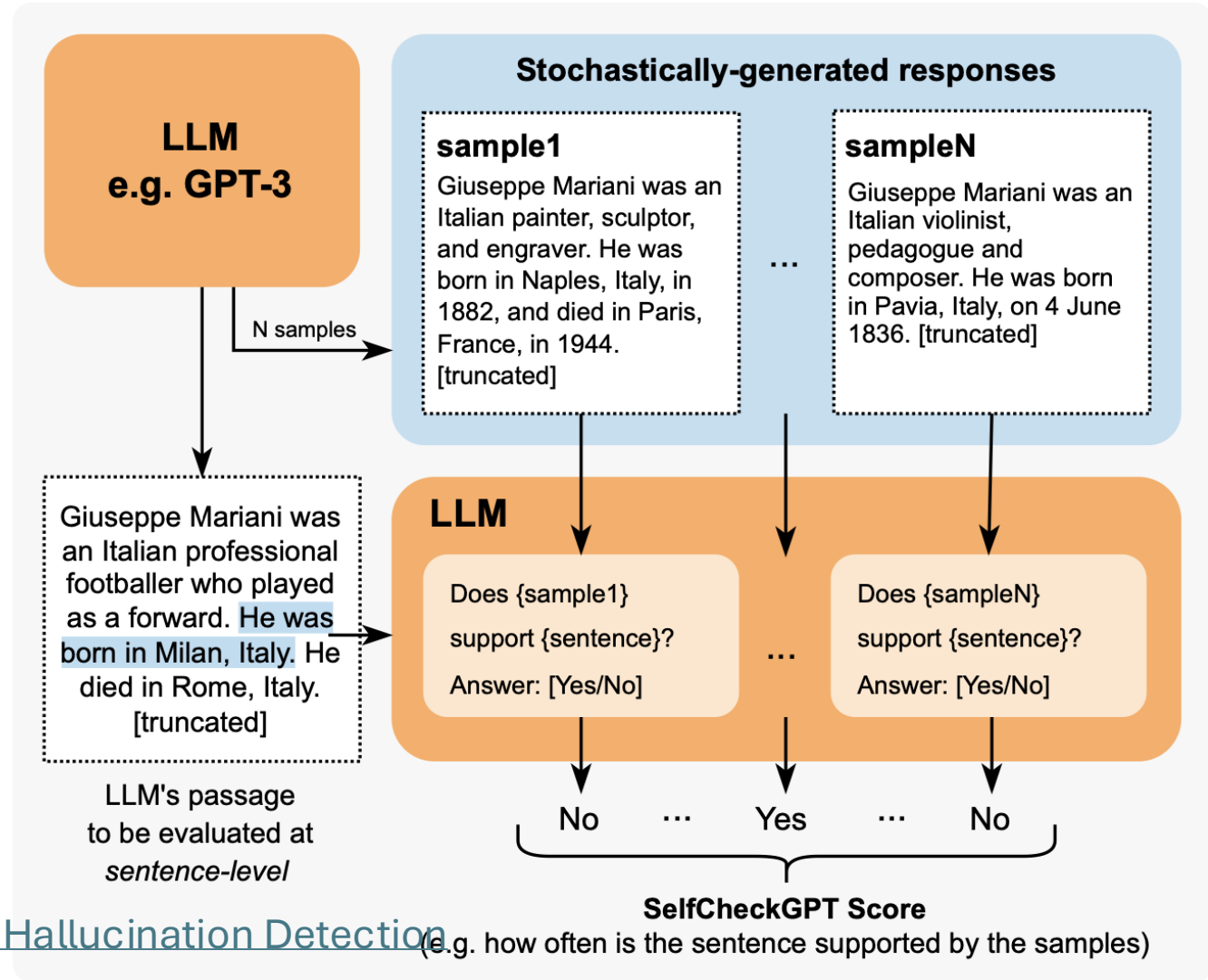
QAG (Question Answer Generation) Score is a scorer that leverages LLMs' high reasoning capabilities to reliably evaluate LLM outputs.

- Use an LLM to extract all claims made in an LLM output.
- For each claim, ask the ground truth whether it agrees ('yes') or not ('no') with the claim made.

Evaluation: QAG Score

QAG (Question Answering)
LLMs' high reasoning

- Use an LLM to extract
- For each claim, ask to support or not ('no') with the claim



[SELFCKGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models](#)

Evaluation: System Level

Latency

- TTFT (time to first token)
- Total latency
- p50 / p95 / p99

Throughput / cost

- tokens/sec (output tokens/sec)
- requests/sec (RPS)
- GPU memory usage / concurrency
- \$/1k tokens (if hosted)

Evaluation: System Level

Reproducibility checklist

Must report:

- hardware (GPU type), precision/quantisation
- batching/concurrency strategy
- prompt/output length distribution
- decoding settings
- warm vs cold runs

Evaluation: System Level

Why system-level metrics are non-negotiable

Same algorithm can look better/worse depending on:

- batching strategy (static/dynamic/continuous)
- prompt length mix
- temperature / decoding method
- hardware memory bandwidth

Evaluation: Take away

- Use **automatic** metrics for correctness when ground truth exists
- Use **human** eval for preference/safety/open-ended quality
- Use **system** eval for real-world performance
- Always match metric $\leftrightarrow$ question $\leftrightarrow$ workload

Next steps

Summarisation: Extractive vs abstractive summarisation

Additional Material:

1. [G-EVAL: NLG Evaluation using GPT-4 with Better Human Alignment](#)
2. [DAG \(Deep Acyclic Graph\)](#)
3. GPTScore: Evaluate as You Desire
4. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.

 r.mu2@exeter.ac.uk

Questions?

